

Week 1: Algorithms Data Structures

Exercise 2 Solution: E-commerce Platform Search Function

1. **Big O notation** is a mathematical tool used to describe how an algorithm's runtime or memory requirements increases as the input data grows.

It generally represents the worst case, and helps us compare algorithms and find out which algorithm performs better for large datasets.

2. **Best Case** Scenario for search operation:

The item is found immediately.

Example:

Products: [101,102,103,104,105]

Search: 101 (Found at first position.)

Time Complexity: $O(1)$

Average Case Scenario for search operation:

The item is somewhere in the middle.

Example:

Products: [101,102,103,104,105]

Search: 103 (Need to check a few elements)

Time Complexity: $O(n)$

Worst Case Scenario for search operation:

The item is at the last position or doesn't exist.

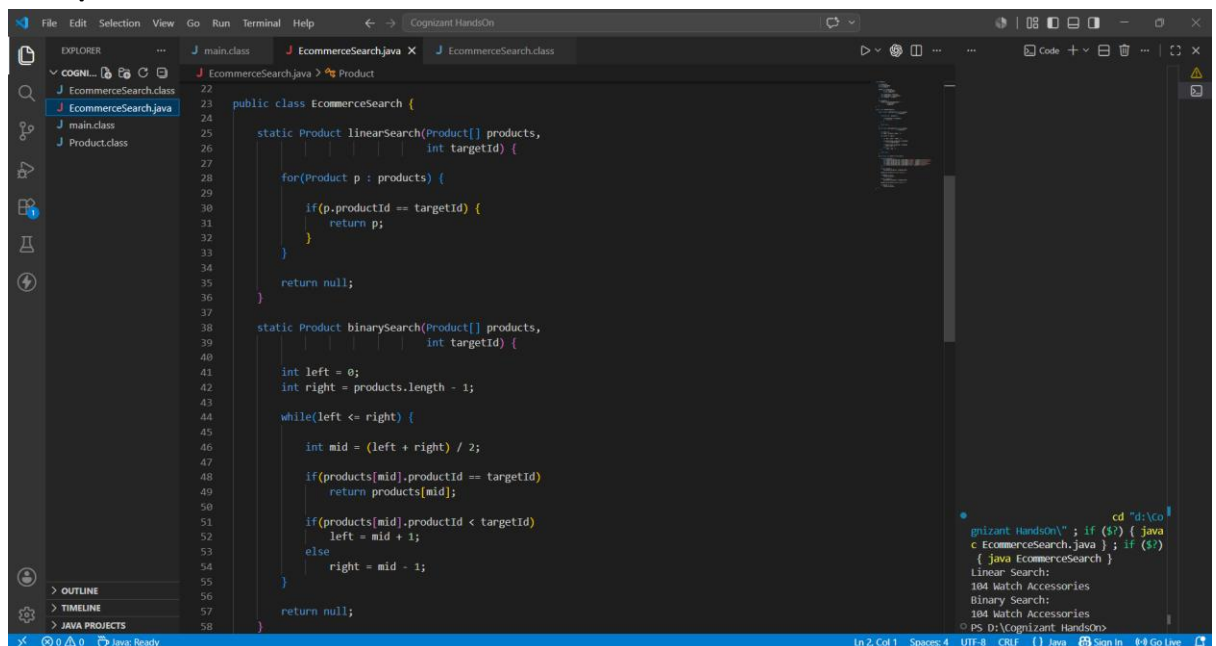
Example:

Products: [101,102,103,104,105]

Search: 105 or 999 (Need to check every element.)

Time Complexity: $O(n)$

3. Output:



```
22
23 public class EcommerceSearch {
24
25     static Product linearSearch(Product[] products,
26                                 int targetId) {
27
28         for(Product p : products) {
29
30             if(p.productId == targetId) {
31                 return p;
32             }
33         }
34
35         return null;
36     }
37
38     static Product binarySearch(Product[] products,
39                                 int targetId) {
40
41         int left = 0;
42         int right = products.length - 1;
43
44         while(left <= right) {
45
46             int mid = (left + right) / 2;
47
48             if(products[mid].productId == targetId)
49                 return products[mid];
50
51             if(products[mid].productId < targetId)
52                 left = mid + 1;
53             else
54                 right = mid - 1;
55         }
56
57         return null;
58     }
59 }
```

The screenshot shows an IDE with a file explorer on the left containing 'EcommerceSearch.class', 'main.class', and 'Product.class'. The main editor displays the Java code for 'EcommerceSearch.java'. The code includes two static methods: 'linearSearch' which iterates through all products, and 'binarySearch' which uses a while loop to find the product by its ID. The status bar at the bottom indicates 'Java: Ready'.

4. For **Linear Search**: Best Case :- $O(1)$

Average Case :- $O(n)$

Worst Case :- $O(n)$

5. For **Binary Search**: Best Case :- $O(1)$

Average Case :- $O(\log n)$

Worst Case :- $O(\log n)$

6. For an ecommerce site, binary search is best because if we need to search 1000 elements, in worst case scenario, linear search will iterate through all 1000 elements. But binary search will roughly make 20 iterations to find the element. That's why Binary Search is best.

Exercise 7 Solution: Financial Forecasting

1. Recursion is basically a program in which a function calls itself until it reaches a stopping condition called base case.

Instead of using loops, we can use recursive techniques as it is faster when working on complex tree/graphs.

Recursion is mainly useful when a problem can be broken down into smaller problems, making it easier to understand and implement.

2. Setup:

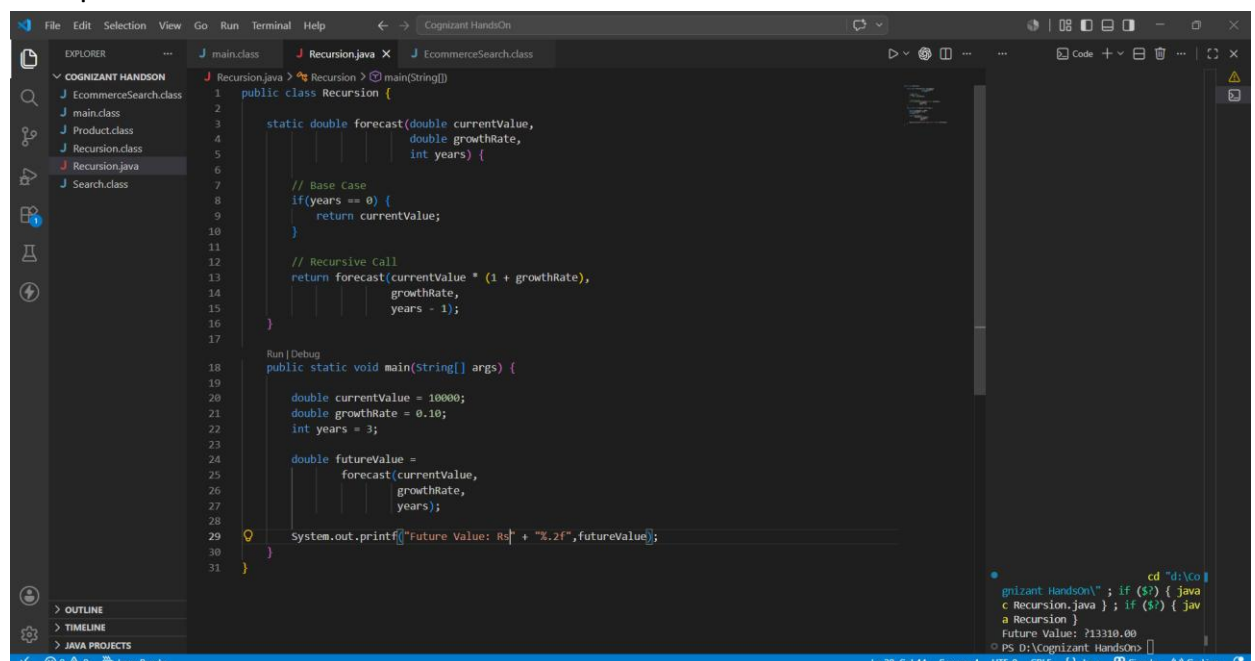
Assume: Current Value: rs 10,000

Growth Rate = 10% per year

Years: 3

Recursive logic : $\text{Future Value} = \text{Current Value} * 1 + \text{Growth Rate}$

3. Output:



The screenshot shows an IDE with a project named 'COGNIZANT HANDSON'. The file explorer on the left lists several files: 'EcommerceSearch.class', 'main.class', 'Product.class', 'Recursion.class', 'Recursion.java', and 'Search.class'. The main editor displays the code for 'Recursion.java'. The code defines a recursive method 'forecast' that calculates the future value based on a current value, growth rate, and number of years. The base case is when years are 0, returning the current value. The recursive call increases the current value by the growth rate and decreases the years by 1. The 'main' method initializes the current value to 10000, growth rate to 0.10, and years to 3, then calls the 'forecast' method and prints the result.

```
1 public class Recursion {
2
3     static double forecast(double currentValue,
4                             double growthRate,
5                             int years) {
6
7         // Base Case
8         if(years == 0) {
9             return currentValue;
10        }
11
12        // Recursive Call
13        return forecast(currentValue * (1 + growthRate),
14                        growthRate,
15                        years - 1);
16    }
17
18    Run | Debug
19    public static void main(String[] args) {
20
21        double currentValue = 10000;
22        double growthRate = 0.10;
23        int years = 3;
24
25        double futureValue =
26            forecast(currentValue,
27                    growthRate,
28                    years);
29
30        System.out.printf("Future Value: Rs" + "%.2f", futureValue);
31    }
32 }
```

The output console at the bottom right shows the command to run the program and the resulting output: 'Future Value: 213310.00'.

4. Analysis:

For time complexity, this method makes one recursive call for each year.

Therefore for n years, time complexity is $O(n)$.

For Space Complexity, each recursive call is stored in memory until the base case is reached.

Space Complexity = $O(n)$.

5. Recursive methods can be optimized by using memorization, storing previously calculated results.

Also, by using a mathematical formula whenever possible.

Or simply by using an iterative loop instead of recursion if the problem doesn't go well with recursion.